1 Is this a retrieval model or a ranking model? What are the outputs?

1.1 Short answer

This repository implements **watch-time prediction** for already-observed (user, item/video) interactions. In that sense, the provided training/evaluation scripts operate as a **pointwise regression / distribution modeling** setup rather than a classic **retrieval** system.

1.2 Retrieval vs. ranking vs. pointwise prediction

- **Retrieval** typically means: given a query/user context, score (often via approximate nearest neighbors) a very large candidate set to retrieve top-K items.
- **Ranking** typically means: given a smaller candidate set, assign scores and sort to produce an ordered list.
- **Pointwise prediction** (what the run scripts do here) means: predict a target value for each example independently (e.g. watch time) and optimize a pointwise loss (NLL, MAE/L1/MSE-like components).

In this repo, the scripts in `run_*.py` iterate over a `DataLoader` of interaction records and predict a watch-time value/distribution for each record. They do not generate candidates, do not compute top-K lists, and do not train with a pairwise/listwise ranking loss.

1.3 What does the model output?

There are two useful notions of "output":

- **Internal outputs** used to define a distribution / loss (e.g. mixture parameters).
- **Final prediction** used for evaluation (a scalar watch-time prediction).

1.4 EGMN outputs

EGMN (`model/egmn.py`) produces parameters of a mixture distribution:

- mixture logits π (converted to mixture weights via softmax),
- exponential rate λ,
- Gaussian parameters μ and σ for multiple components.

The evaluation code uses a **scalar point prediction** computed as the **mixture mean**.

```
# run_egmn.py (illustrative structure)
with torch.no_grad():
    for features, label in dataloaders["test"]:
        y = model.predict(features) # scalar per example
        # collect y for MAE / XAUC / KL

# model/egmn.py (illustrative)
pi, lambda_, mu, sigma = self.forward(features)
pi = torch.softmax(pi, dim=1)
```

```
component_means = torch.concat([1 / lambda_, mu], dim=1)
pred = torch.sum(pi * component_means, dim=1)
```

1.5 Baseline outputs (examples)

Different baselines output different intermediate objects, but they all ultimately provide a scalar prediction for evaluation:

Wide & Deep (`WideAndDeep`) Outputs a **single probability-like scalar** (sigmoid of an MLP + optional linear part). In this repo, it is used as a pointwise predictor.

CREAD (`Cread`) Outputs a **vector of bucket/head probabilities**. The training script discretizes the label into multiple binary thresholds and trains with BCE. At test time it reconstructs a **scalar watch-time estimate** from the bucket outputs.

D2Q (`D2Q`) Outputs a **normalized quantile score** (sigmoid). The training script maps labels into bucket-specific quantiles, trains with MSE, then maps predicted quantiles back to values.

```
# Examples of different "output shapes"

# Wide&Deep: scalar
p = wide_and_deep(features) # shape: [batch]

# CREAD: vector over M bucket heads
bucket_probs = cread(features) # shape: [batch, M]

# D2Q: scalar quantile (later mapped back to value)
q = d2q(features) # shape: [batch]
```

1.6 Is XAUC a ranking metric?

The repo reports **XAUC**, which is computed by sorting examples by predicted score and measuring agreement with label ordering. This is a **ranking-style evaluation metric**, but it does not mean the models are trained with a retrieval/ranking objective; training is still pointwise (NLL/regression-style) in the provided scripts.